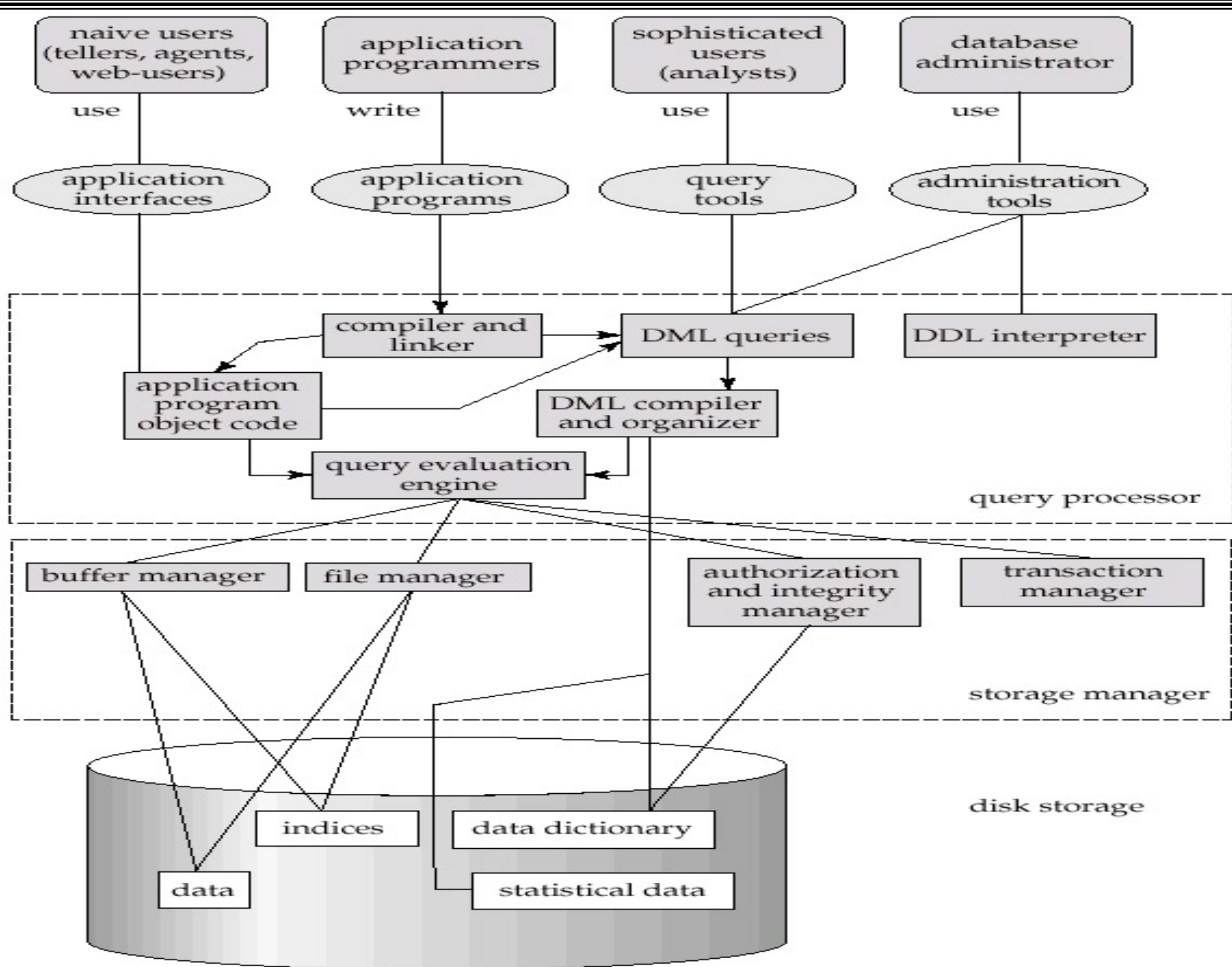


Chapter 13

Data Storage Structures





- - Storage Access

- - File Organization

- - Organization of Records in Files

- - Data-Dictionary Storage



■ A database file is partitioned into fixed-length storage units called **blocks** (块) . Blocks are units of both storage allocation and data transfer.

■ Database system seeks to minimize the number of block transfers between the disk and memory.

keep as many blocks as possible in main memory.

■ **Buffer** – portion of main memory available to store copies of disk blocks.

■ **Buffer manager** – subsystem responsible for allocating buffer space in main memory.



- ★ If the block is already in the buffer, the requesting program is given the address of the block in main memory
- ★ If the block is not in the buffer,
 1. the buffer manager allocates space in the buffer for the block, replacing (throwing out) some other block, if required, to make space for the new block.
 2. The block that is thrown out is written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
 3. Once space is allocated in the buffer, the buffer manager reads the block from the disk to the buffer, and passes the address of the block in main memory to requester.



Buffer-Replacement Policies *DataBase System Concepts*

- Most operating systems replace the block **least recently used** (LRU strategy)

- Queries have well-defined access patterns (such as sequential scans), and a database system can use the information in a user's query to predict future references

- ★ LRU can be a bad strategy for certain access patterns involving repeated scans of data

- ★ e.g. when computing the join of 2 relations r and s by a nested loops

- for each tuple tr of r do

- for each tuple ts of s do

- if the tuples tr and ts match ...



- **■Pinned block(钉住的块)** – memory block that is not allowed to be written back to disk.

- ✓Pin done before reading/writing data from a block

- ✓Unpin done when read /write is complete

- ✓Multiple concurrent pin/unpin operations possible

- **■forced output (强制写出)**

- **■Toss-immediate (立即丢弃)** strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed



- Most recently used (MRU) strategy –

- Buffer manager can use statistical information regarding the probability that a request will reference a particular relation

 - ★ E.g., the data dictionary is frequently accessed.

 - Heuristic: keep data-dictionary blocks in main memory buffer

- Buffer managers also support forced output of blocks for the purpose of recovery



Shared and exclusive locks on buffer

- Needed to prevent concurrent operations from reading page contents as they are moved/reorganized, and to ensure only one move/reorganize at a time.
- Readers get shared lock, updates to a block require exclusive lock

Locking rules:

- Only one process can get exclusive lock at a time
- Shared lock cannot be concurrently with exclusive lock
- Multiple processes may be given shared lock concurrently



- The database is stored as a collection of *files*. Each file is a sequence of *records*. A record is a sequence of fields;
- Files are provided as a basic construct in OS;
- The records are mapped onto disk blocks;
 - Fixed-length records
 - Variable-length records



■ Simple approach:

- Store record i starting from byte $n * (i - 1)$, where n is the size of each record.
- Record access is simple but records may cross blocks

Modification: do not allow records to cross block boundaries

■ Deletion of record i : alternatives:

- move records $i + 1, \dots, n$ to $i, \dots, n - 1$
- move record n to i
- do not move records, but link all free records on a *free list*



record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



Deleting record 3 and compacting

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

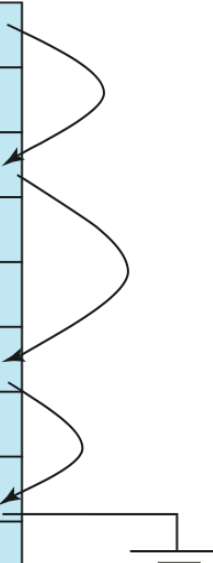
Deleting record 3 and moving last record

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

- Store the address of the first deleted record in the file header.
- Use this first record to store the address of the second deleted record, and so on.
- Can think of these stored addresses as pointers since they “point” to the location of a record.
- More space efficient representation: reuse space for normal attributes of free records to store pointers. (No pointers stored in in-use records.)



header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000



■ Variable-length records arise in database systems in several ways:

- Storage of multiple record types in a file.
- Record types that allow variable lengths for one or more fields.
- Record types that allow repeating fields

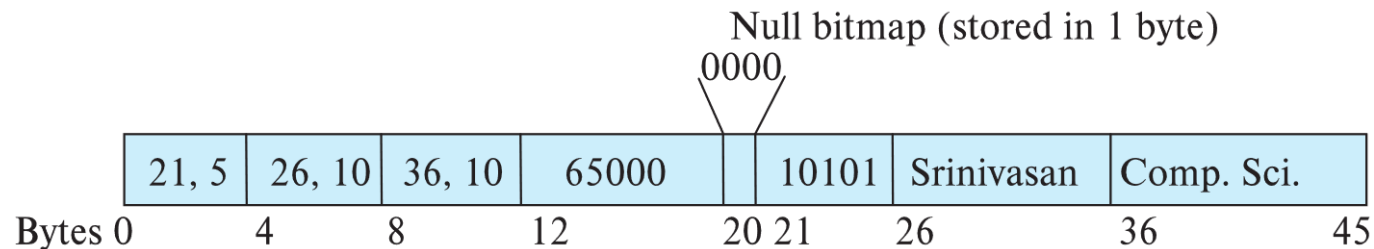
■ Byte string representation

- Attach an *end-of-record* ($\perp$) control character to the end of each record
- Difficulty with deletion
- Difficulty with growth

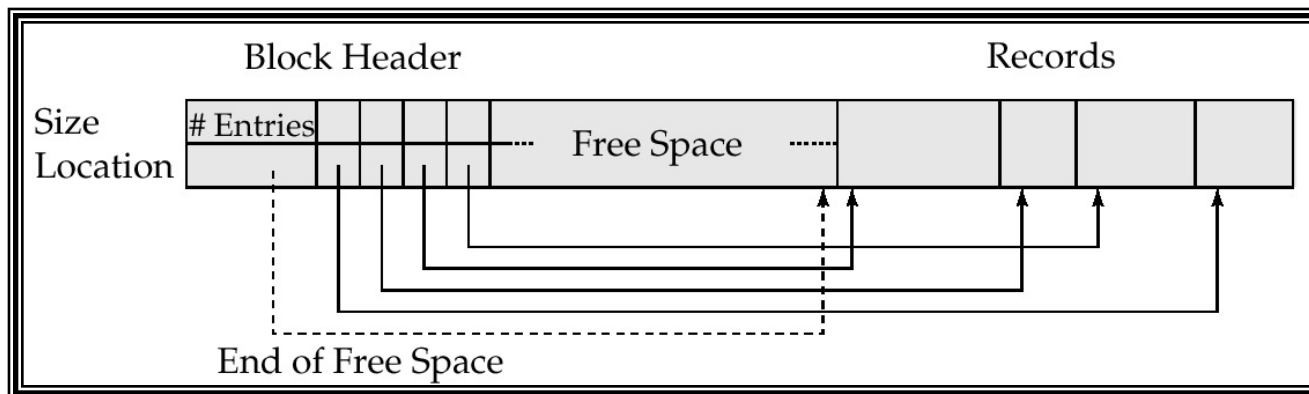


Variable-Length Records

- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap



Slotted Page Structure



- Slotted page header contains:
 - number of record entries
 - end of free space in the block
 - location and size of each record
- Records can be moved around within a page to keep them contiguous with no empty space between them; entry in the header must be updated.
- Pointers should not point directly to record — instead they should point to the entry for the record in header.

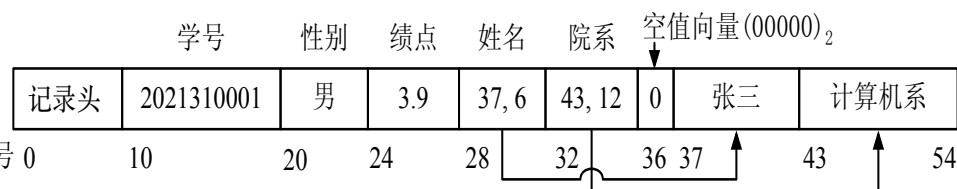


记录

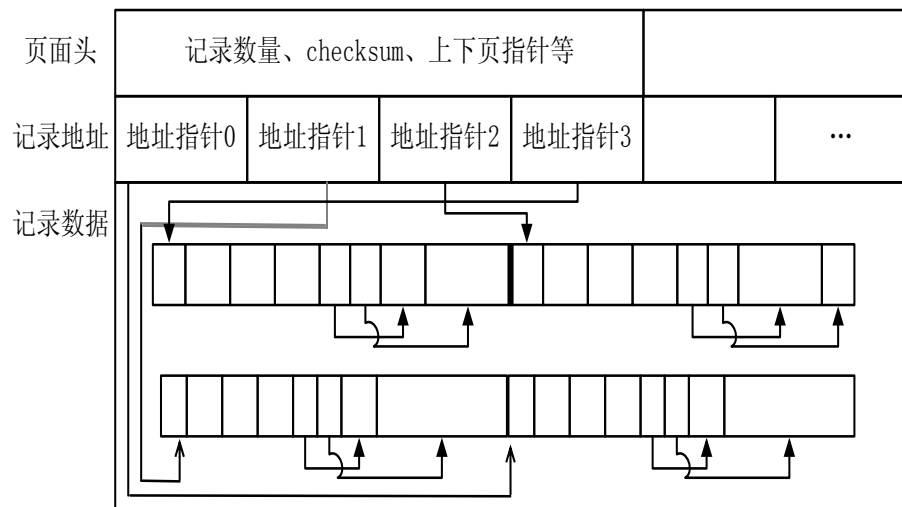
2021310001	张三	男	计算机系	3.9
------------	----	---	------	-----

Char Varchar Char Varchar Decimal

记录的字节表示



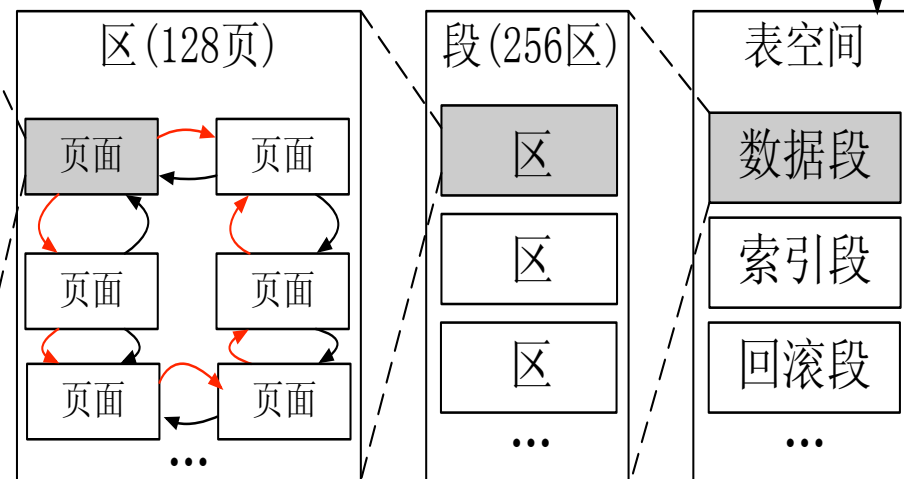
分槽页面 (8K)



数据表

	学号	姓名	性别	院系	绩点
记录0	2021310001	张三	男	计算机系	3.9
记录1	2021310002	张四	女	计算机系	3.7
记录2	2021310003	张五	男	计算机系	4.0
记录3	2021310004	张六	男	法学系	3.7
记录4	2021310006	张七	女	物理系	3.4
记录5	2021310009	张八	女	物理系	3.6
记录6	2021310013	张九	女	法学系	3.7

文件



- E.g., blob/clob types
- Records must be smaller than pages
- Alternatives:
 - Store as files in file systems
 - Store as files managed by database
 - Break into pieces and store in multiple tuples in separate relation
 - PostgreSQL TOAST



- **Heap** (堆) – a record can be placed anywhere in the file where there is space
- **Sequential** (顺序) – store records in sequential order, based on the value of the search key of each record
- **Hashing** (哈希) – a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed
- **B+-tree file organization**
- Records of each relation may be stored in a separate file. In a **clustering file organization** (聚集) records of several different relations can be stored in the same file
 - Motivation: store related records on the same block to minimize I/O



- Records can be placed anywhere in the file where there is free space
- Records usually do not move once allocated
- Important to be able to efficiently find free space within file
- **Free-space map**

- Array with 1 entry per block. Each entry is a few bits to a byte, and records fraction of block that is free
- In example below, 3 bits per block, value divided by 8 indicates fraction of block that is free

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Can have second-level free-space map
- In example below, each entry stores maximum from 4 entries of first-level free-space map

4	7	2	6
---	---	---	---



Sequential File Organization

- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a search-key
- Deletion – use pointer chains
- Insertion –locate the position where the record is to be inserted
 - If there is free space insert there
 - if no free space, insert the record in an overflow block
 - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order



Insertion Example

10101	Srinivasan	Comp. Sci.	65000		
12121	Wu	Finance	90000		
15151	Mozart	Music	40000		
22222	Einstein	Phys	10101	Srinivasan	Comp. Sci. 65000
32343	El Said	Histo	12121	Wu	Finance 90000
33456	Gold	Phys	15151	Mozart	Music 40000
45565	Katz	Com	22222	Einstein	Physics 95000
58583	Califieri	Histo	32343	El Said	History 60000
76543	Singh	Finar	33456	Gold	Physics 87000
76766	Crick	Biolo	45565	Katz	Comp. Sci. 75000
83821	Brandt	Com	58583	Califieri	History 62000
98345	Kim	Elec.	76543	Singh	Finance 80000
			76766	Crick	Biology 72000
			83821	Brandt	Comp. Sci. 92000
			98345	Kim	Elec. Eng. 80000
32222	Verdi	Music	48000		



Multitable Clustering File Organization DataBase System Concepts

■ store several relations in one file using a **clustering** file organization

department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering
of *department* and
instructor

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

- good for queries involving *department* *instructor*, and for queries involving one single department and its instructors
- bad for queries involving only *department*
- results in variable size records
- Can add pointer chains to link records of a particular relation

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	




- **Data dictionary** (also called system catalog) stores metadata: that is, data about data, such as
 - Information about relations
 - names of relations
 - names and types of attributes of each relation
 - names and definitions of views
 - integrity constraints
 - User and accounting information, including passwords
 - Statistical and descriptive data
 - number of tuples in each relation



- Physical file organization information
 - How relation is stored (sequential/hash/...)
 - Physical location of relation
 - operating system file name or
 - disk addresses of blocks containing records of the relation
- Information about indices (Chapter 15)



■Catalog structure: can use either

- specialized data structures designed for efficient access
- a set of relations, with existing system features used to ensure efficient access

The latter alternative is usually preferred

Relation-metadata = (relation-name, number-of-attributes,
storage-organization, location)

Attribute-metadata = (attribute-name, relation-name, domain-
type, position, length)

User-metadata = (user-name, encrypted-password, group)

Index-metadata = (index-name, relation-name, index-type,
index-attributes)

View-metadata = (view-name, definition)



Relational Representation of System Metadata

DataBase System Concepts

- Relational representation on disk
- Specialized data structures designed for efficient access, in memory

Relation_metadata
<u>relation_name</u>
number_of_attributes
storage_organization
location

Index_metadata
<u>index_name</u>
<u>relation_name</u>
index_type
index_attributes

View_metadata
<u>view_name</u>
definition

Attribute_metadata
<u>relation_name</u>
<u>attribute_name</u>
domain_type
position
length

User_metadata
<u>user_name</u>
encrypted_password
group



Column-Oriented Storage

- Also known as **columnar representation**
- Store each attribute of a relation separately
- Example: instructor

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califleri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000



Column-Oriented Storage

- Benefits:
 - Reduced IO if only some attributes are accessed
 - Improved CPU cache performance
 - Improved compression
 - **Vector processing** on modern CPU architectures
- Drawbacks
 - Cost of tuple reconstruction from columnar representation
 - Cost of tuple deletion and update
 - Cost of decompression
- Columnar representation found to be more efficient for decision support than row-oriented representation
- Traditional row-oriented representation preferable for transaction processing
- Some databases support both representations



Storage Organization in Main-Memory Databases

DataBase System Concepts

- Can store records directly in memory without a buffer manager
- Column-oriented storage can be used in-memory for decision support applications
 - Compression reduces memory requirement



- Buffer Management and replacement policies

- File organization

 - Variable-length records

 - Heap file, Sequential file

 - Hashing file , Clustering file

- Data dictionary Storage

- Column-Oriented Storage

